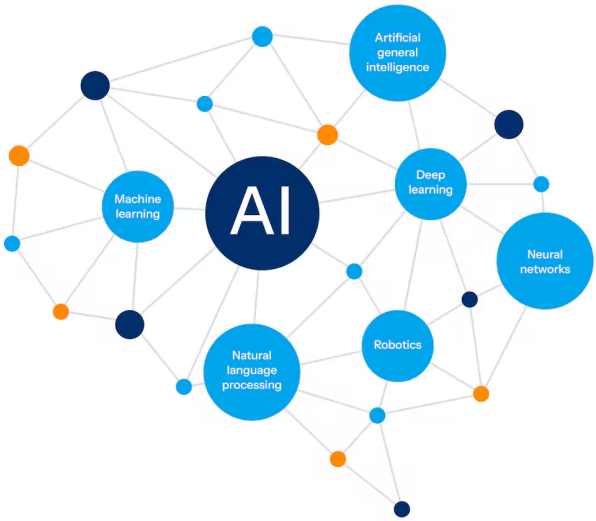


The Complete NLP Cheat Sheet

From raw text to transformers explained clearly with examples.

Save this. Every AI/ML engineer needs this.

BASICS

What is NLP?

Natural Language Processing teaches machines to read, understand, and generate human language.

WHAT NLP POWERS TODAY

- Search engines
- Translation tools
- Voice assistants
- AI chatbots
- Spam filters
- Sentiment analysis
- Summarization

THE CORE CHALLENGE

```
"I saw the man with the telescope"

Who has the telescope?
→ The man?
→ The person watching?
```

Human language is ambiguous.
Machines must handle this at scale.

STEP 1

Text Preprocessing Part 1

Before any model sees your text, it needs cleaning. This step is often more important than the model itself.

STEP 1: LOWERCASING

```
"The Quick Brown FOX"
↓
"the quick brown fox"
```

STEP 2: REMOVING NOISE

```
"Hello!! Visit site.com"
↓
"Hello Visit"
```

STEP 3: PUNCTUATION

```
"Wait, really? Yes!"
↓
"Wait really Yes"
```

STEP 4: STOP WORDS

```
"the cat sat on the mat"
↓
"cat sat mat"
```

Stop words are common words (the, on, is) that carry little meaning.

STEP 1

Text Preprocessing Part 2

STEP 5: STEMMING

Chops word endings. Fast but crude.

```
"running" → "run"          "happiness" → "happi"
"connection" → "connect"   "studies" → "studi"
```

STEP 6: LEMMATIZATION

Converts to true dictionary base form. Slower but accurate.

```
"running" → "run"          "better" → "good"
"geese" → "goose"          "studies" → "study"
```

STEMMING VS LEMMATIZATION

✕ Stemming "studies" → "studi" (Not a real word)

✓ Lemmatization "studies" → "study" (Real word)

STEP 2

Tokenization

Splitting text into units the model can process. The very first step in every NLP pipeline.

WORD

```
"I love NLP"
↓
["I", "love", "NLP"]
```

SENTENCE

```
"Hi. Bye."
↓
["Hi.", "Bye."]
```

SUBWORD (USED BY TRANSFORMERS)

```
"unhappiness" → ["un", "happi", "ness"]
"ChatGPT"     → ["Chat", "G", "PT"]
```

WHY SUBWORD?

- ✓ Handles Typos ("teh")
- ✓ New words ("COVID")
- ✓ Multiple languages

COMMON TOKENIZERS

- BPE → GPT-4
- WordPiece → BERT
- tiktoken → OpenAI

STEP 3

Text Representation Part 1

Machines cannot read words. They read numbers. Here are the main ways text becomes numbers.

METHOD 1: ONE HOT ENCODING

```
cat  → [1, 0, 0]
dog  → [0, 1, 0]
```

✕ Problem: No relationship between words. Very sparse.

METHOD 2: BAG OF WORDS (BOW)

```
"the cat sat on the cat" → {the:2, cat:2, sat:1, on:1}
```

✕ Problem: Ignores word order completely. ("dog bites man" = "man bites dog")

METHOD 3: TF-IDF

Words common across docs score low. Unique words score high.

```
TF = word count / total words  |  IDF = log(total docs / docs with word)
TF-IDF = TF × IDF
```

STEP 3

Text Representation Part 2

METHOD 4: WORD EMBEDDINGS

Words mapped to dense vectors. Similar words are close.

```
king  → [0.82, 0.14, 0.73]
queen → [0.79, 0.12, 0.71]

king - man + woman ≈ queen ✓
```

✕ Problem: Same word = same vector regardless of context.

METHOD 5: CONTEXTUAL EMBEDDINGS

Same word gets a different vector based on context. (BERT, GPT)

```
"I went to the bank to deposit money."
bank → vector_A (financial)

"We sat by the river bank."
bank → vector_B (river side)
```

EVOLUTION

One-hot → BoW → TF-IDF → Word2Vec → BERT/GPT

DEEP DIVE

Word Embeddings Part 1

The foundation of modern NLP. Each word is a point in high-dimensional space.

SPATIAL RELATIONSHIP

Word2Vec (2013)

CBOW

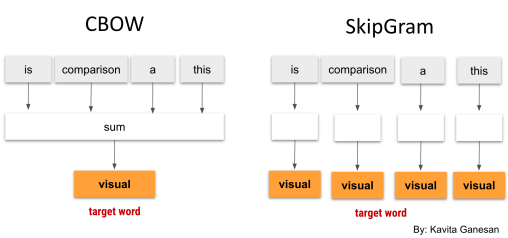
Predict target from context

```
"The ___ sat"
→ predicts: "cat"
```

SKIP-GRAM

Predict context from target

```
"cat"
→ predicts: "sat", "the"
```



Skip-gram is better for rare words. CBOW is faster.

DEEP DIVE

Word Embeddings Part 2

GLOVE (2014)

Trains on global word co-occurrence statistics. Better at analogies.

```
"ice" co-occurs with "solid", "cold"
"steam" co-occurs with "gas", "hot"

ice - water + gas ≈ steam ✓
```

FASTTEXT (2016) BY META

Trained on character subwords. Handles rare words.

```
"unhappy" uses subwords:
"un", "unh", "nha", "hap", "app"...

Can handle words NEVER seen in training ✓
```

DIMENSIONS

- GloVe: 50–300
- BERT: 768
- GPT-3: 12,288

GRAMMAR

Part of Speech Tagging

Assigns a grammatical label to every word.

COMMON TAGS

```
NN → Noun (dog)          VB → Verb (run)
JJ → Adjective (fast)    RB → Adverb (quickly)
PRP → Pronoun (I, he)    IN → Preposition (in)
```

EXAMPLE

```
"The quick brown fox jumps"
The   → DT (Determiner)
quick → JJ (Adjective)
fox   → NN (Noun)
jumps → VBZ (Verb)
```

Why it matters: Resolves ambiguity.
"Book (VB) a table" vs "Read a book (NN)"

EXTRACTION

Named Entity Recognition

NER finds and classifies real-world entities in text.

ENTITY TYPES

- PERSON (Sundar Pichai)
- ORG (Google)
- GPE (India)
- DATE (2024)
- MONEY ($1B)

IOB TAGGING

How NER is trained:

- **B-** Beginning of entity
- **I-** Inside entity
- **O** Outside entity

EXAMPLE

```
"Sundar Pichai joined Google in 2004"

Sundar → B-PERSON
Pichai → I-PERSON
joined → O
Google → B-ORG
in     → O
2004   → B-DATE
```

EMOTION

Sentiment Analysis

Classifying the emotional tone behind text.

LEVELS OF SENTIMENT

Document: "Product is amazing." → Positive

Sentence: "Food great but service bad." → Mixed

Aspect: "Camera excellent, battery poor."

THREE APPROACHES

Rule-based: Word lists (fails on sarcasm)

ML-based: Naive Bayes, SVM

Transformer: BERT (handles nuance) ✓

WHY CONTEXT MATTERS

```
"Not bad at all."
Rule-based sees "bad" → Negative ✗
BERT reads context    → Positive ✓
```

ORGANIZATION

Text Classification

Assigning predefined categories. Very common in production.

TYPES

Binary: Spam / Not Spam

Multi-class: Sports / Tech

Multi-label: [NLP, AI, Tech]

METHODS

BoW + Naive Bayes: ~75%

TF-IDF + LR: ~85%

BERT: ~95%+

LLM (Zero-shot): No training

KEY METRICS

```
Accuracy  = correct / total
Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
F1 Score  = 2 × (P × R) / (P + R)
```

Balanced dataset → Accuracy. Imbalanced → F1 Score.

PREDICTION

Language Modeling

Predicts the probability of the next word. Foundation of AI models.

CORE IDEA

```
P("I love NLP") =
P(I) × P(love | I) × P(NLP | I love)
```

N-GRAM MODELS (CLASSICAL)

Bigram: predicts from 1 previous word

Trigram: predicts from 2 previous words

✕ Problem: Fails on long-range dependencies.

NEURAL MODELS

→ **RNN:** forgets over long sequences

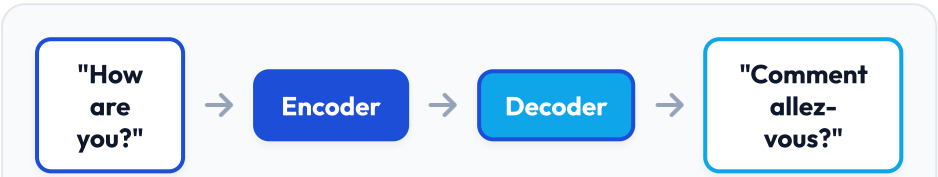
→ **LSTM:** better memory, still limited

✓ **Transformer:** attends to full sequence at once

ARCHITECTURE

Sequence to Sequence

Architecture where both input and output are variable-length.



PROBLEM WITH BASIC SEQ2SEQ

Entire input is compressed into ONE fixed vector.

Short input ("Hi") → works fine

Long input (500 words) → information lost ✕

SOLUTION: ATTENTION

Decoder looks at **ALL** encoder hidden states and weights each one differently per output token. Nothing is lost. ✓

THE CORE

Attention Mechanism

Lets the model focus on the most relevant parts of the input.

Q, K, V INTUITION

Query (Q): What am I looking for?

Key (K): What does each token offer?

Value (V): What is actually passed forward?

$$\text{Score} = \text{softmax}(Q \times K^T / \sqrt{d}) \times V$$

MULTI-HEAD

Run attention H times in parallel. Each head learns different relationships (syntactic, semantic, etc.).

TYPES

Self: same sequence

Cross: decoder to encoder

Masked: hides future

MODERN NLP

Transformer Part 1

"Attention Is All You Need" (2017). Replaced RNNs.

✕ RNN

Processes sequentially

Forgets early tokens

Slow to train

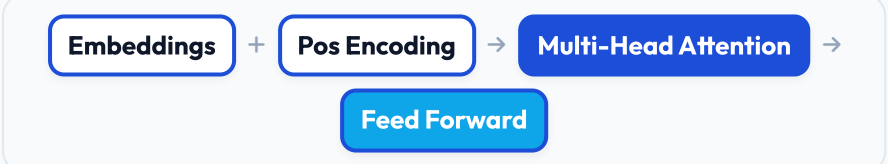
✓ TRANSFORMER

Processes in parallel

Attends to full sequence

Fast to train

ENCODER BLOCK (UNDERSTANDS)



Positional Encoding: Since tokens process in parallel, this adds position info so word order isn't lost.

MODERN NLP

Transformer Part 2

THREE TRANSFORMER VARIANTS

Encoder only:	(BERT, RoBERTa)	Sees full context. Best for: classification, NER, QA.
Decoder only:	(GPT, LLaMA)	Left to right. Best for: generation, chat, code.
Encoder + Decoder:	(T5, BART)	Best for: translation, summarization.

STACK DEPTH

- BERT base: 12 layers
- BERT large: 24 layers
- GPT-2: 12 layers
- GPT-3: 96 layers

COMPARISON

BERT vs GPT

Two of the most important model families. They solve different problems.

BERT

Direction: Bidirectional

Training: Masked Language Modeling (Predicts missing word)

Architecture: Encoder only

✓ Classification, NER, QA

✗ NOT for text generation

GPT

Direction: Left to right

Training: Next token prediction

Architecture: Decoder only

✓ Generation, chat, code

✗ NOT for bidirectional understanding

WHICH TO USE?

Sentiment → BERT

NER Tagging → BERT

Summarization → GPT

Chatbot → GPT

MEASUREMENT

Evaluation Metrics

FOR CLASSIFICATION

Accuracy: Use when classes are balanced.

Precision: How often positive prediction is right.

Recall: How many actual positives are caught.

F1 Score: Use when classes are imbalanced.

FOR GENERATION

BLEU Score: Translation. Measures word overlap.

ROUGE Score: Summarization. (unigram, bigram overlap).

BERTScore: Uses embeddings to compare meaning. Better than BLEU.

Perplexity: Language modeling. Lower is better.

</> TOOLKIT

NLP Libraries Reference

SPACY

Best for: Production pipelines

Speed: Very fast

NLTK

Best for: Learning, research

Speed: Slower

HF TRANSFORMERS

Best for: State-of-the-art models (BERT, GPT, LLaMA)

SENTENCE TRANSF.

Best for: Semantic similarity, embeddings

GENSIM

Best for: Word2Vec, topic modeling (LDA)